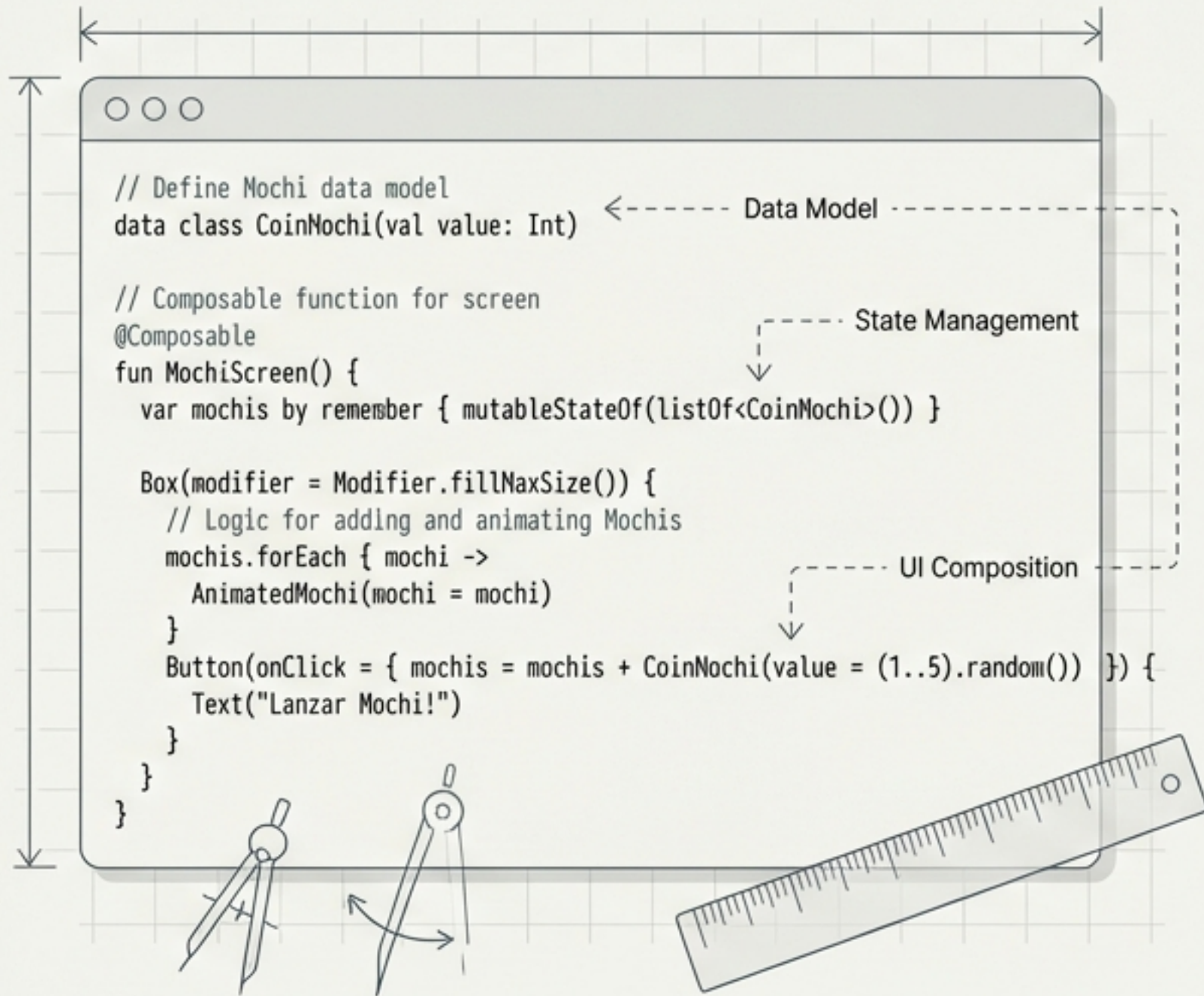


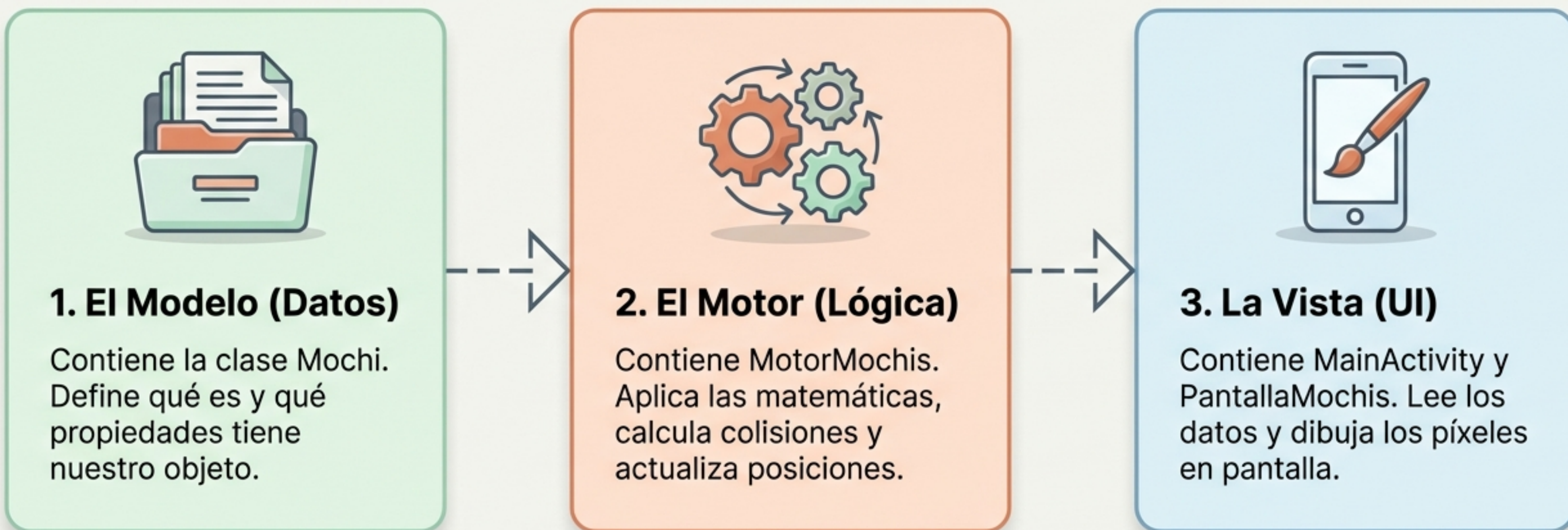
De Código a Pantalla: Creando tu Primera App Interactiva

Una guía visual para entender Kotlin, Jetpack Compose y la arquitectura moderna de Android paso a paso.



La Arquitectura del Proyecto: Dividiendo para Conquistar

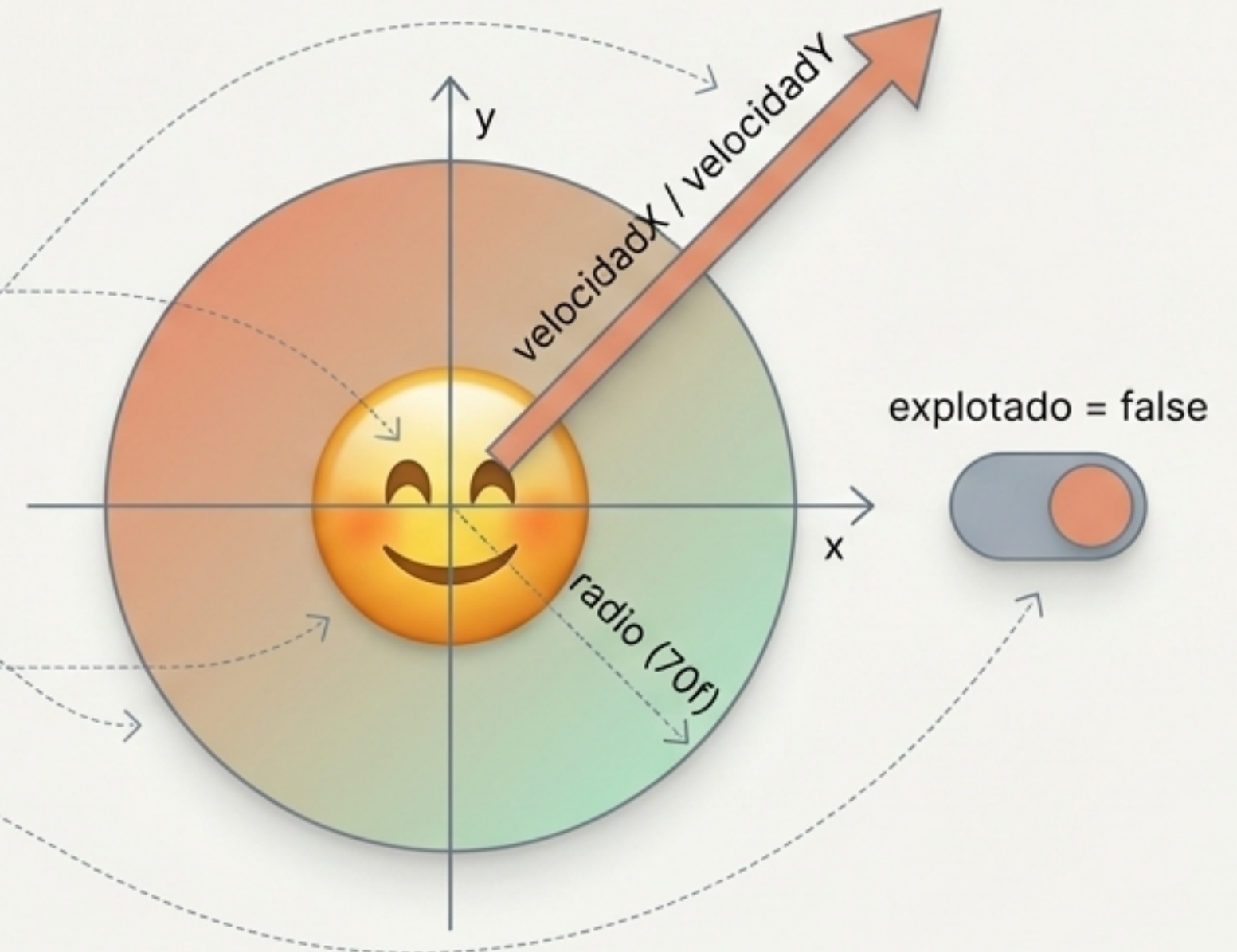
Para construir una app escalable, separamos las responsabilidades. El Modelo guarda la información, el Motor aplica las reglas físicas, y la Interfaz Gráfica dibuja el resultado final.



El ADN del Juego: Definiendo la Entidad Principal

En Kotlin, una data class es el molde perfecto. No ejecuta acciones complejas; su único propósito es almacenar el estado exacto de un objeto en un momento dado. Aquí definimos qué hace a un Mochi ser un Mochi.

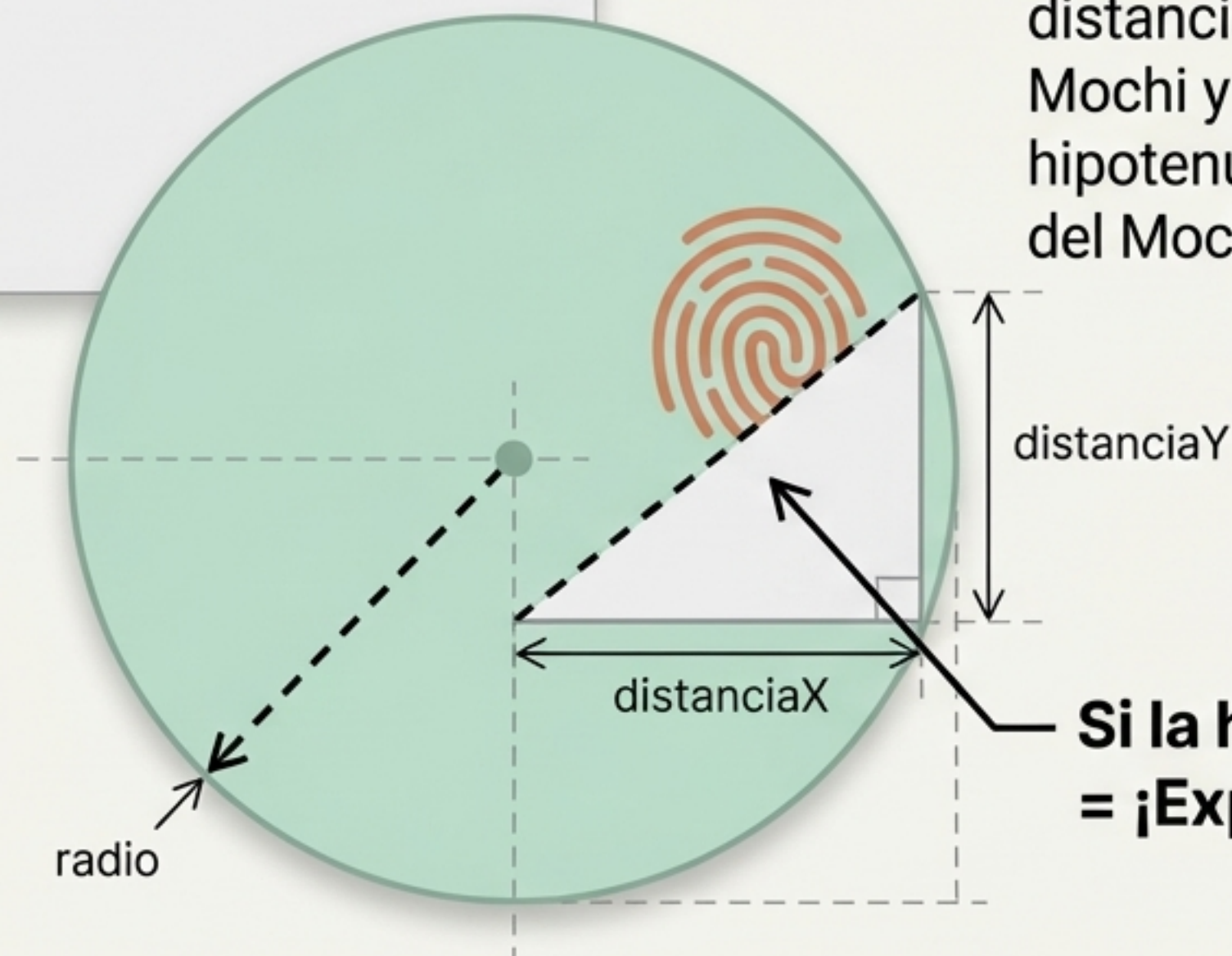
```
data class Mochi(  
    var x: Float,  
    var y: Float,  
    var velocidadY: Float = 0f,  
    var velocidadX: Float = 0f,  
    val radio: Float = 70f,  
    var emoji: String,  
    var explotado: Boolean = false  
)
```



Matemáticas en Código: ¿Cómo Sabe un Objeto si lo Tocaste?

```
fun fueTocado(xDelDedo: Float, yDelDedo: Float): Boolean {  
    val distanciaX = xDelDedo - x  
    val distanciaY = yDelDedo - y  
    // Teorema de Pitágoras aquí  
}
```

Calculamos la colisión usando geometría básica. Encontramos la distancia exacta entre el centro del Mochi y tu dedo. Si esa distancia (la hipotenusa) es menor que el radio del Mochi, el toque fue un éxito.

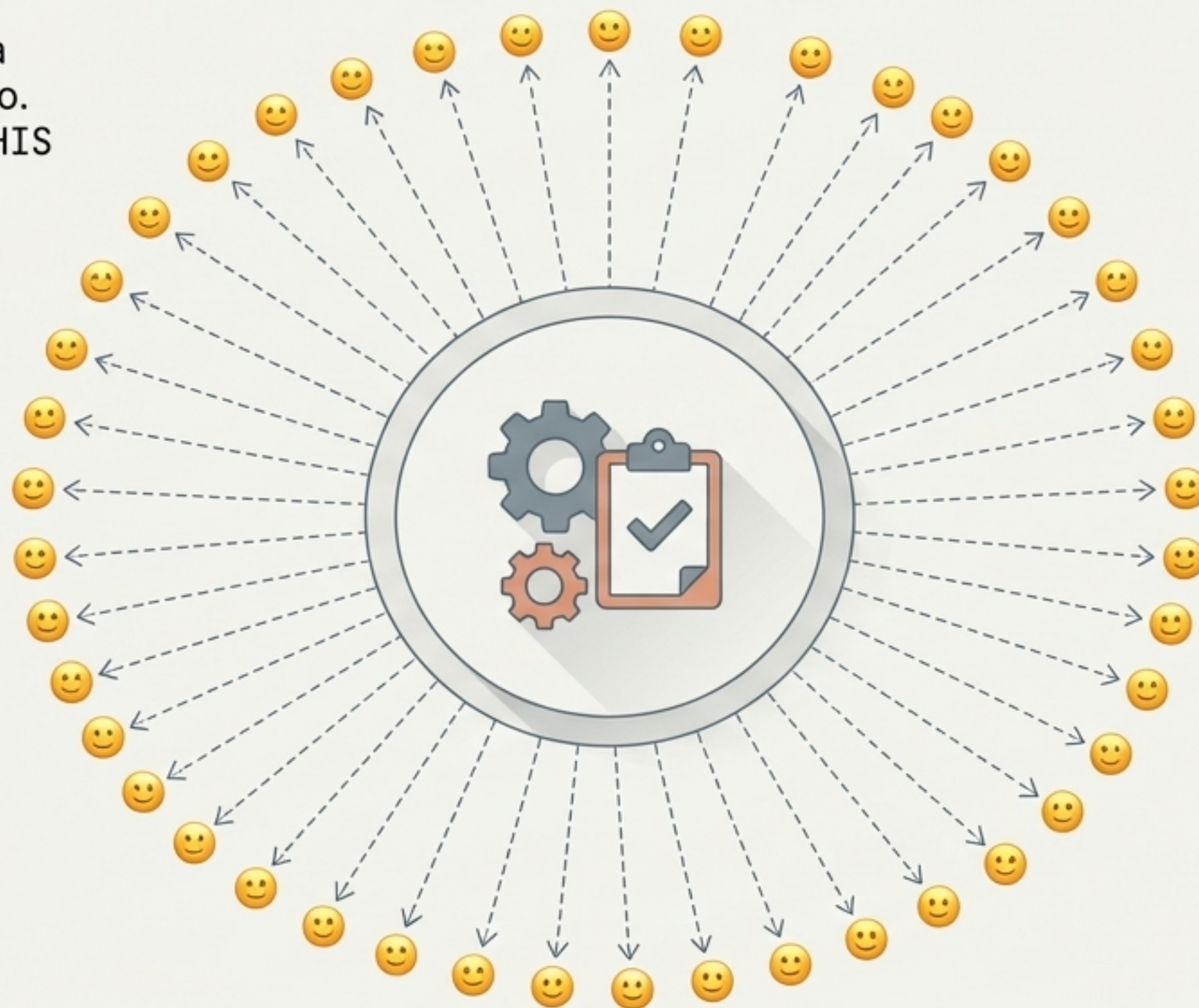


**Si la hipotenusa < radio
= ¡Explotado!**

El Cerebro de la Operación: Controlando el Caos

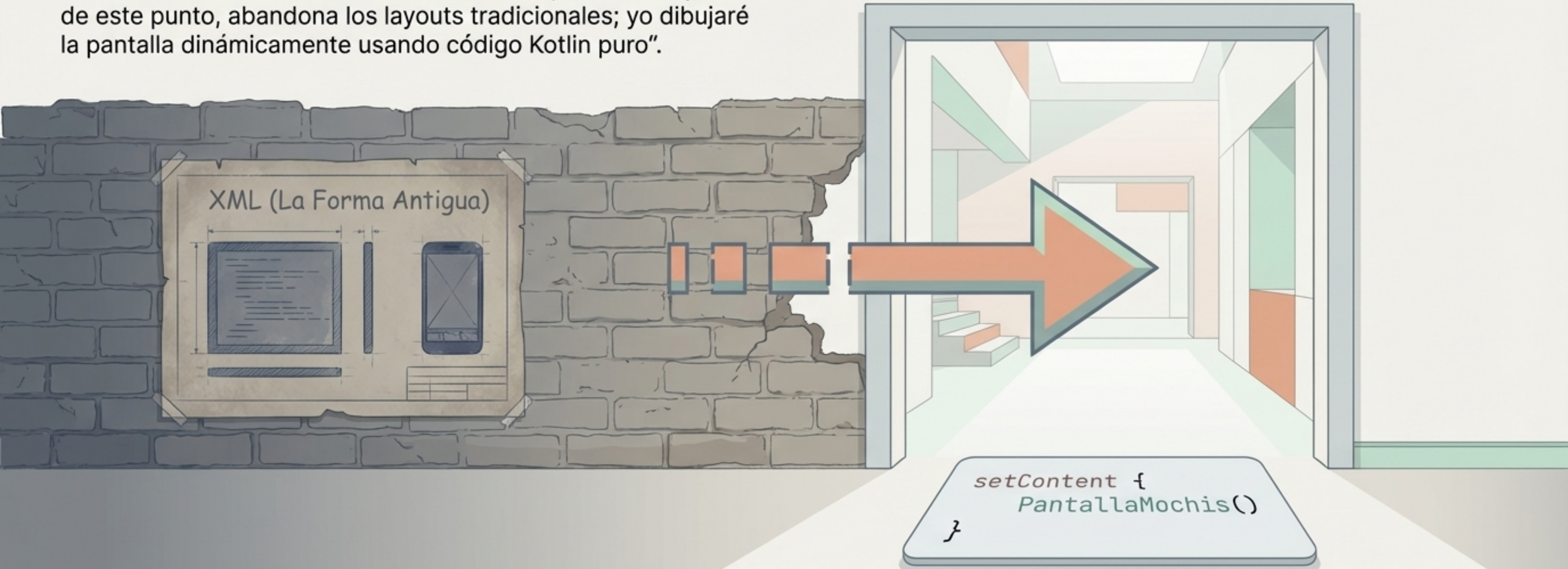
El MotorMochis es el director de la orquesta. Controla la colección entera y dicta las reglas físicas del mundo. Usamos un companion object para declarar MAX_MOCHIS como una constante global: una regla inquebrantable de nuestra app.

```
class MotorMochis {  
  companion object {  
    const val MAX_MOCHIS = 1000  
  }  
}
```



Entrando al Mundo Visual de Jetpack Compose

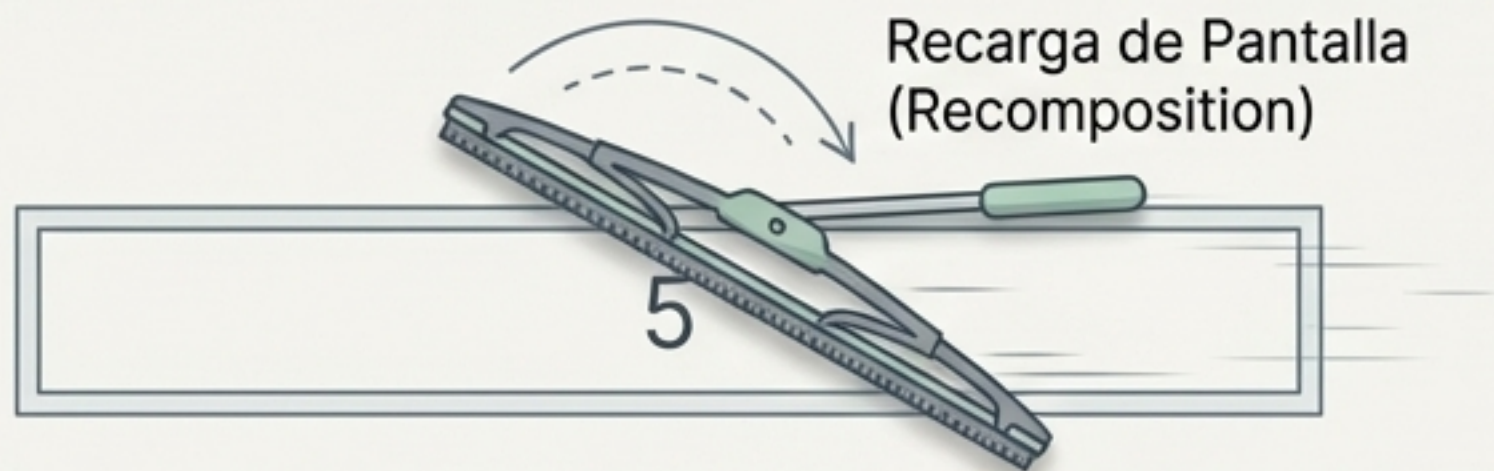
MainActivity es tu puerta de entrada al sistema Android. Al llamar a `setContent`, le decimos al sistema operativo: "A partir de este punto, abandona los layouts tradicionales; yo dibujaré la pantalla dinámicamente usando código Kotlin puro".



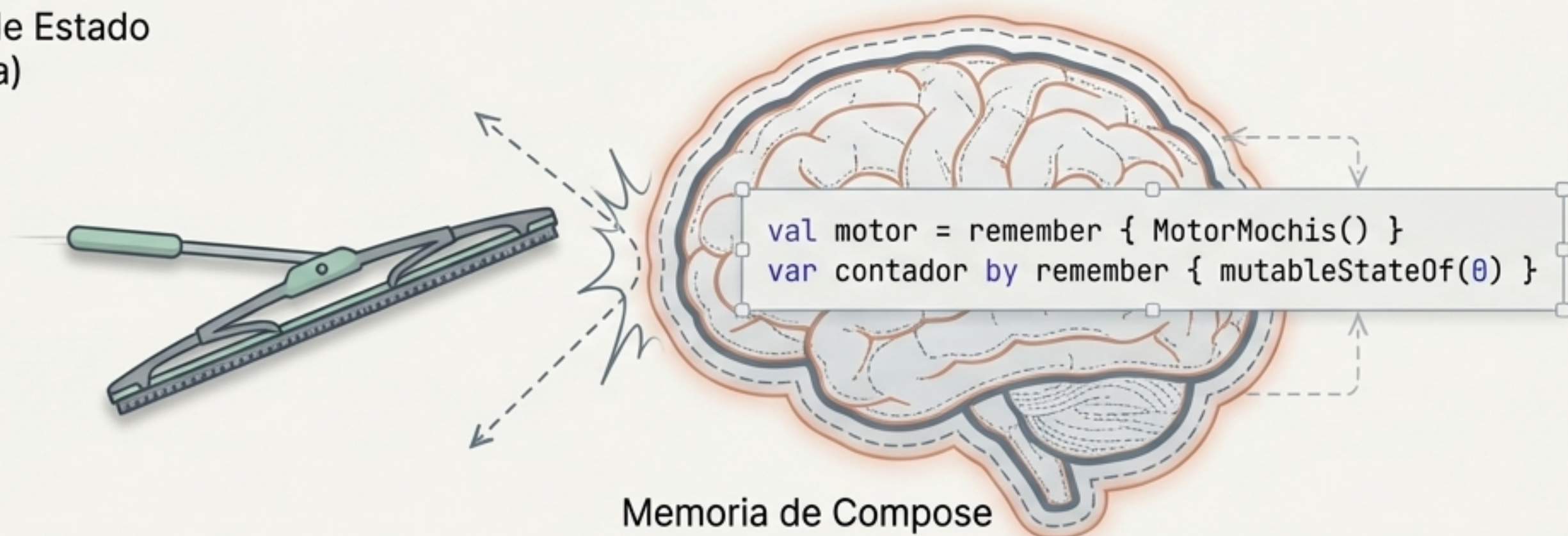
La Magia del Estado: La Memoria a Corto Plazo de la Interfaz

En Compose, la pantalla se redibuja constantemente. Las variables normales se olvidan en cada ciclo. `remember` y `mutableStateOf` actúan como la memoria protectora de la interfaz, asegurando que los datos sobrevivan a las actualizaciones visuales.

Variable Normal
(Sin protección)

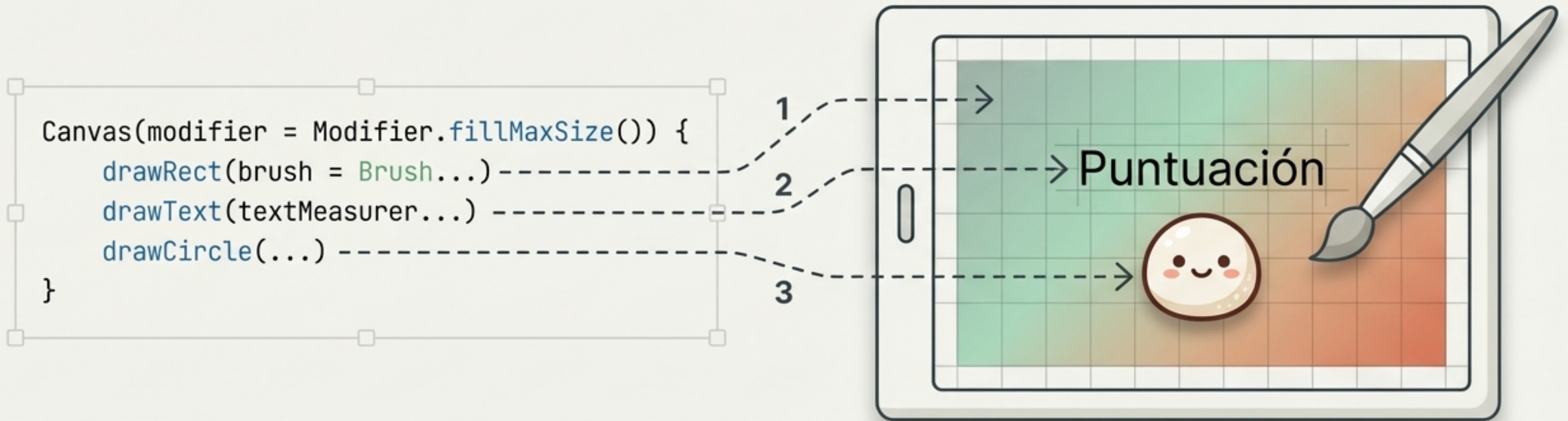


Variable de Estado
(Protegida)



El Lienzo Mágico: Control Total sobre Cada Píxel

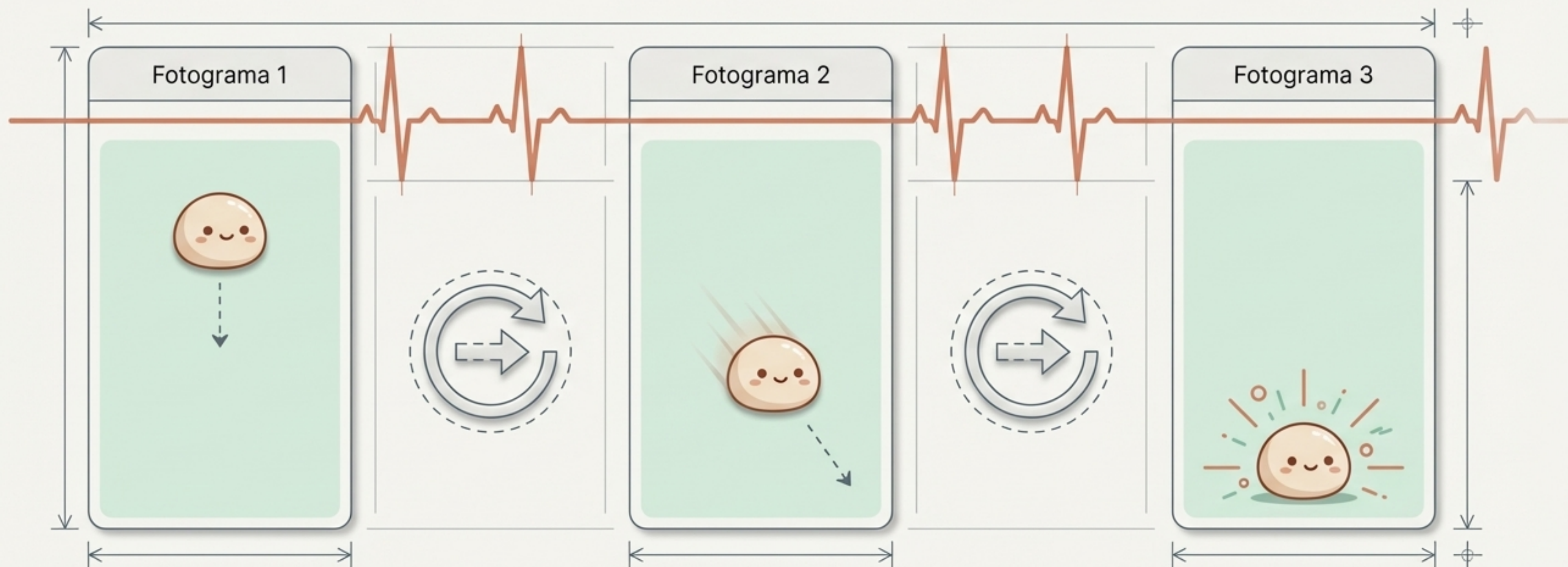
Compose ofrece botones prefabricados, pero para juegos necesitamos Canvas. Es una herramienta de bajo nivel que nos da el poder absoluto para pintar fondos dinámicos, dibujar texto a medida y renderizar cada Mochi frame por frame.



El Latido del Juego: Animación Continua

```
val animacion =  
rememberInfiniteTransition()
```

Los juegos no son imágenes estáticas, son ilusiones creadas por actualizaciones rápidas. `rememberInfiniteTransition` actúa como el latido infinito del sistema, obligando al motor a recalcular físicas y pedirle a la pantalla que se redibuje continuamente.



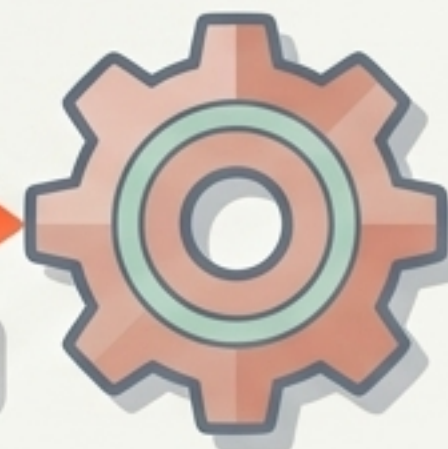


Interactividad: Escuchando los Dedos del Usuario

El lienzo no solo dibuja; también escucha. Con `detectTapGestures`, capturamos coordenadas exactas de la pantalla. Al tocar, la interfaz le envía una alerta inmediata al Motor para que verifique si el usuario logró hacer explotar algún Mochi.

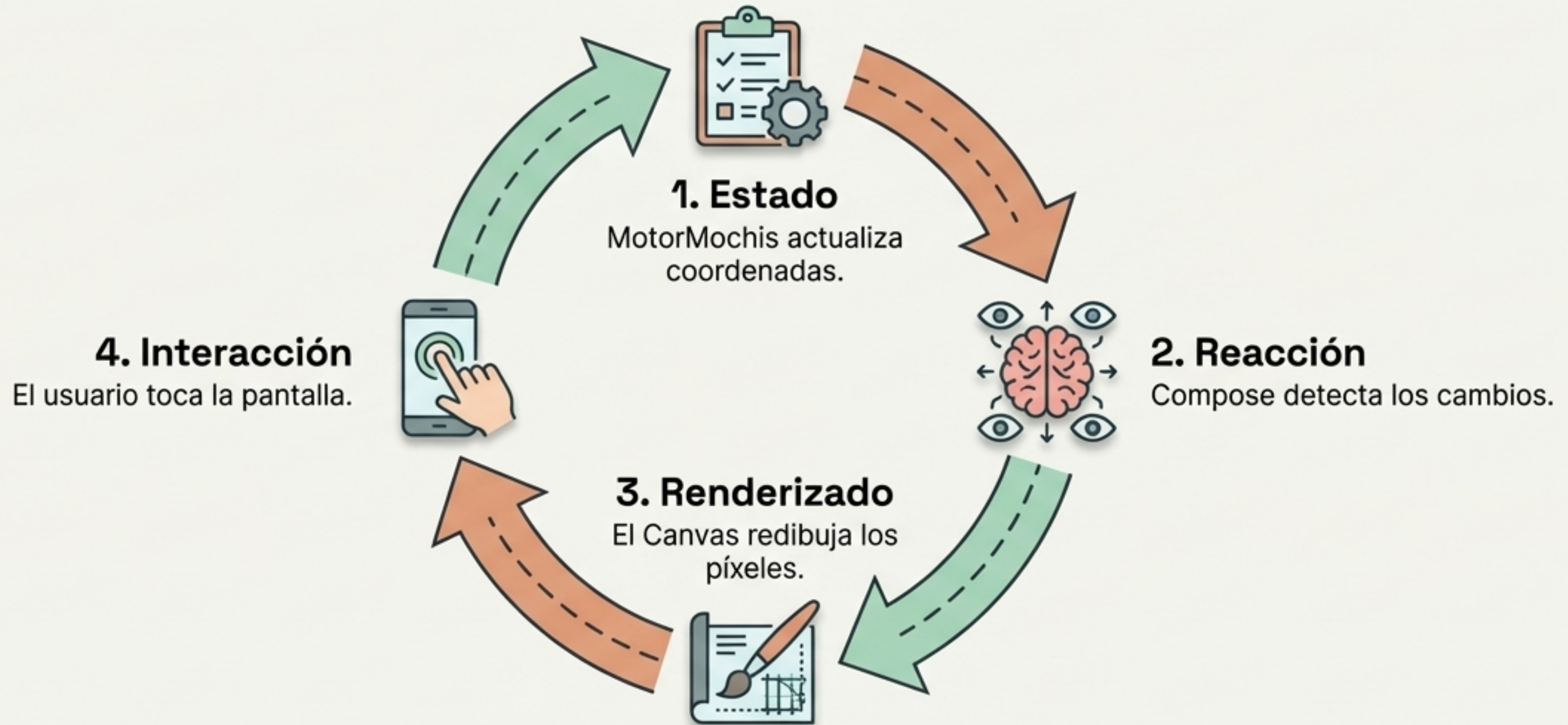
```
Modifier.pointerInput(Unit) {  
    detectTapGestures { tap ->  
        // Captura coordenadas (x, y)  
    }  
}
```

Alerta: ¡Revisar colisiones en X, Y!



El Bucle de Recomposición: Cómo Funciona el Android Moderno

Este es el gran secreto de Jetpack Compose. Modificas los datos (Estado), y la interfaz reacciona automáticamente para dibujarlos. Tú te enfocas en la lógica; Compose se encarga de la pintura.



Tu Nuevo Kit de Herramientas para Interfaces Dinámicas

Has deconstruido un motor interactivo completo. Estas cuatro herramientas forman la base de cualquier aplicación moderna, desde animaciones simples hasta interfaces de producción masiva.

Herramienta	Su Propósito Principal	Nuestro Caso de Uso
 <code>data class</code>	Almacena datos inmutables o mutables de una entidad.	El ADN físico del Mochi.
 <code>State / remember</code>	Memoria a corto plazo de Compose.	Proteger el motor durante el redibujado.
 <code>Canvas</code>	Dibujo libre sobre una cuadrícula de píxeles.	Renderizar colores y emojis fotograma a fotograma.
 <code>pointerInput</code>	Receptor de gestos y coordenadas físicas.	Detectar toques precisos para explotar Mochis.